EC2 instances types.

1. **General Purpose Instance**

These provide a **balanced mix of CPU, memory, and networking**.

Examples

- M series (M5, M6i)
- T series (T2, T3, T4g)
- A1

Use cases

- Web servers
- Small databases
- Dev/test environments
- Microservices

Reality:
Most beginners use **T2 or T3** because they are cheap and burstable.

2. **Compute Optimized Instances**

These provide **high CPU power**.

Examples

- C5
- C6i
- C7g

Use cases

- High performance computing
- Gaming servers
- Batch processing
- Media transcoding

If your application is **CPU-heavy**, using general instances is inefficient.

3. Memory Optimized Instances

These provide **large RAM capacity**.

Examples

- R5 / R6i
- X1 / X2
- z1d

Use cases

- Large databases
- In-memory caching
- Real-time big data analytics

If the application constantly loads large datasets, you need this.

4. Storage Optimized Instances

Designed for **high disk throughput and IOPS**.

Examples

- I3 / I4
- D2
- H1

Use cases

- Big data processing
- Data warehouses
- Log processing systems

5. **Accelerated Computing Instances**

Uses **GPU or FPGA hardware acceleration**.

Examples

- P series (machine learning training)
- G series (graphics / video rendering)
- F series (FPGA)

Use cases

- Machine learning
- AI model training
- Video rendering
- 3D graphics

AWS CLI connection

1. What is AWS CLI

The **AWS Command Line Interface (CLI)** is a command-line tool that enables users to interact with AWS services through commands.

Using AWS CLI, users can create, manage, and monitor resources such as:

- Amazon EC2 instances
- Amazon S3 buckets
- Amazon Elastic Kubernetes Service clusters
- AWS Identity and Access Management users and roles

It is widely used in **automation, scripting, and DevOps workflows**.

2. Purpose of AWS CLI Connection

The main purpose of connecting AWS CLI to an AWS account is to:

- Manage AWS services from a terminal
- Automate infrastructure operations

- Integrate with CI/CD pipelines
- Execute AWS commands quickly without logging into the AWS console

This improves efficiency and allows infrastructure to be managed programmatically.

3. Authentication Process

AWS CLI connection requires authentication using **security credentials**.

The authentication is done through:

- **Access Key ID**
- **Secret Access Key**
- **Region configuration**
- **Output format**

```
HARI@LAPTOP-6OHJK66L MINGW64 ~
$ aws --version
aws-cli/2.34.4 Python/3.13.11 Windows/11 exe/AMD64

HARI@LAPTOP-6OHJK66L MINGW64 ~
$
```

```
HARI@LAPTOP-6OHJK66L MINGW64 ~
$ aws configure
AWS Access Key ID [****************5BFK]:
AWS Secret Access Key [****************a5XO]:
Default region name [ap-south-1]:
Default output format [None]: json

HARI@LAPTOP-6OHJK66L MINGW64 ~
$ aws sts get-caller-identity
{
    "UserId": "089665204192",
    "Account": "089665204192",
    "Arn": "arn:aws:iam::089665204192:root"
}
```

```
MINGW64:/c/Users/HARI
HARI@LAPTOP-6OHJK66L MINGW64 ~
$ aws ec2 describe-instances
{
    "Reservations": [
        {
            "ReservationId": "r-084e49e7b7d63bfe5",
            "OwnerId": "089665204192",
            "Groups": [],
            "Instances": [
                {
                    "Architecture": "x86_64",
                    "BlockDeviceMappings": [
                        {
                            "DeviceName": "/dev/sda1",
                            "Ebs": {
                                "AttachTime": "2026-01-31T07:13:32+00:00",
                                "DeleteOnTermination": true,
                                "Status": "attached",
                                "VolumeId": "vol-02041139f76eb0e17",
                                "EbsCardIndex": 0
                            }
                        }
                    ],
                    "ClientToken": "548d2a48-b358-4b3e-a38a-3d308363fca8",
                    "EbsOptimized": true,
                    "EnaSupport": true,
```

This is the creation of the EC2 instance using the CLI.

aws ec2 run-instances \

--image-id <ami-xxxxxxxx> \

--instance-type <type of instances> \

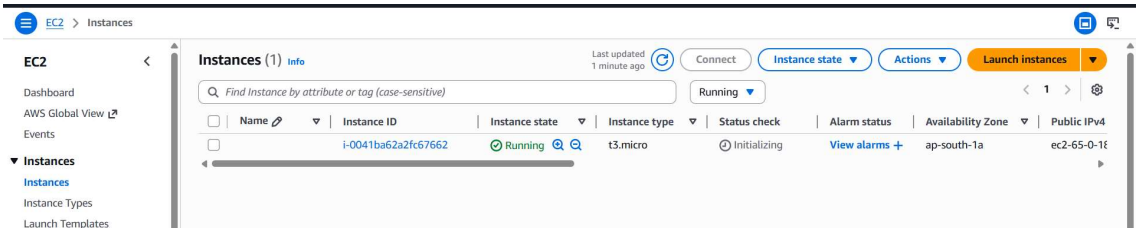
--key-name <key name> \

--security-group-ids <security group name> \

--count 1

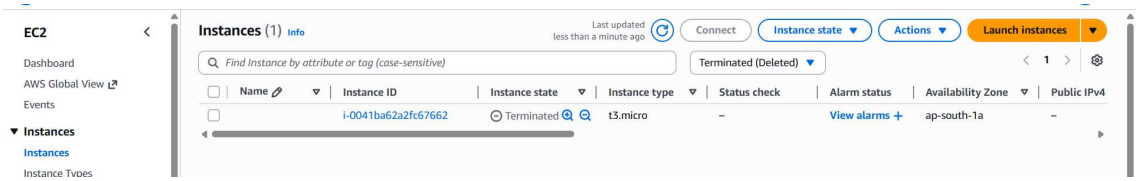
Creation of ec2 instance.

```
MINGW64:/c/Users/HARI

HARI@LAPTOP-60HJK66L MINGW64 ~
$ aws ec2 run-instances --image-id ami-0f559c3642608c138 --instance-type t3.micro --co
unt 1 --region ap-south-1
{
    "ReservationId": "r-0b9271c53e516857f",
    "OwnerId": "089665204192",
    "Groups": [],
    "Instances": [
        {
            "Architecture": "x86_64",
            "BlockDeviceMappings": [],
            "ClientToken": "94035a07-82ac-4a8e-a644-3c6a751c2905",
            "EbsOptimized": false,
            "EnaSupport": true,
            "Hypervisor": "xen",
            "NetworkInterfaces": [
                {
                    "Attachment": {
                        "AttachTime": "2026-03-07T13:55:50+00:00",
                        "AttachmentId": "eni-attach-0379e5325d4efd505",
                        "DeleteOnTermination": true,
                        "DeviceIndex": 0,
                        "Status": "attaching",
                        "NetworkCardIndex": 0
```



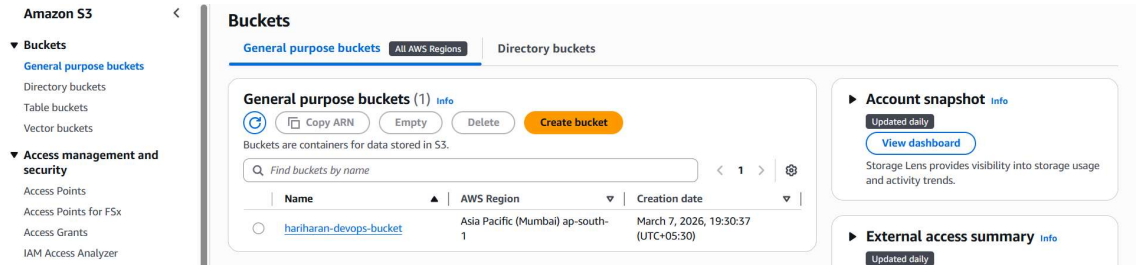
Deletion of EC2 instances.

```
MINGW64:/c/Users/HARI
$

HARI@LAPTOP-60HJK66L MINGW64 ~
$ aws ec2 terminate-instances \
--instance-ids i-0041ba62a2fc67662 \
--region ap-south-1
{
    "TerminatingInstances": [
        {
            "InstanceId": "i-0041ba62a2fc67662",
            "CurrentState": {
                "Code": 32,
                "Name": "shutting-down"
            },
            "PreviousState": {
                "Code": 16,
                "Name": "running"
            }
        }
    ]
}

HARI@LAPTOP-60HJK66L MINGW64 ~
$ |
```



Creation of S3 Bucket.

```
HARI@LAPTOP-60HJK66L MINGW64 ~
$ aws s3 mb s3://hariharan-devops-bucket --region ap-south-1
make_bucket: hariharan-devops-bucket
```



And the removal of S3 Bucket.

```
HARI@LAPTOP-60HJK66L MINGW64 ~
$ aws s3 rb s3://hariharan-devops-bucket
remove_bucket: hariharan-devops-bucket
```